



Lecture 50: Array Updates in \$state vs Legacy Re-assignment

One of the most common stumbling blocks for developers learning Svelte 4 was updating an array. If you had a list of items and you called `items.push(newItem)`, the array would update in memory, but the user interface would completely ignore the change. The new item simply would not appear on screen. This failure occurred because legacy Svelte's reactivity system was fundamentally tied to the **assignment operator** — the literal `=` sign in your code. If the compiler did not see an `=` sign targeting the `items` variable, it assumed the variable had not changed, and therefore no UI update was scheduled. To fix this, developers were forced to write repetitive, non-intuitive boilerplate like `items = [...items, newItem]` or `items = items` simply to force the compiler to notice the mutation.

Svelte 5 eliminates this entire class of bugs by abandoning assignment-based tracking in favor of the `$state` rune. When you pass an array into `$state()`, Svelte does not just hold onto the array; it transforms it into a deeply reactive **state proxy** — a native JavaScript object wrapper that intercepts every time you read or write a property underneath it. Because the proxy intercepts operations at runtime rather than relying on compile-time source code analysis, Svelte 5 inherently understands when you call mutating array methods like `.push()`, `.pop()`, or `.splice()`. When the proxy detects that the array has been mutated through one of these methods, it automatically alerts the reactivity system to surgically update the specific parts of the UI that depend on that array.

● ● ● App.svelte

```
1 <script>
2 let items = $state([1, 2]);
3
4 function addItem() {
5   items.push(3);
6 }
7 </script>
```

Intercept array method mutations **AUTOMATICALLY** at runtime

This shift from compile-time assignment tracking to runtime proxies drastically simplifies how you work with complex data structures. In Svelte 5, you write standard, idiomatic JavaScript. To add an item to an array, you just call `items.push(newItem)`. To update a specific item deep inside an array of objects, you simply access it by index and reassign its property, such as `todos[0].done = true`. Because `$state` creates deep proxies by default, every nested object and array inside your root state is also recursively converted into a proxy, ensuring that even deeply nested mutations are caught and tracked automatically without requiring full object replacement.

● ● ● App.svelte

```
1 <script>
2 let todos = $state([ { text: 'Buy milk', done: false } ]);
3
4 function completeTask() {
5   todos[0].done = true;
6 }
7 </script>
```

Deeply nested mutations are **CAUGHT** without replacement

However, the proxy-based system does introduce a specific constraint regarding destructuring. If you destructure a primitive value out of a state proxy, such as `let { done } = todos[0]`, you are pulling out the raw boolean value at that exact moment in time, completely severing its connection to the reactive proxy. If you later execute `todos[0].done = true`, your local `done` variable will not magically update, because standard JavaScript passes primitive values by value, not by reference. Therefore, when working with `$state` arrays, you must avoid destructuring primitive properties if you need them to remain reactive over time; instead, reference the proxy properties directly in your templates and logic.



App.svelte

```
1  <script>
2  let todos = $state([ { text: 'Buy milk', done: false } ]);
3
4  function toggle() {
5    let { done } = todos[0];
6    done = true;
7  }
8  </script>
```

Pulls raw primitive, severing CONNECTION to proxy

Original proxy is NOT UPDATED